МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА
Факультет електроніки та комп'ютерних технологій

ЗВІТ
З ЛАБОРАТОРНОЇ РОБОТИ № 4

Варіант 6

Рекурсія та мемоізація в Python

Виконав: студент 2 курсу групи ФЕП-22
Мазан Давид

Перевірив: аспірант
Остріков Данило Андрійович

Львів - 2026

Мета роботи

Вивчити рекурсивні алгоритми, базові випадки й кроки розбиття задач, обмеження стека викликів CPython та мемоізацію через functools.lru_cache. Реалізувати quicksort, обходи бінарного дерева, Fibonaccі та завдання варіанта 6 для вкладених списків і кортежів чисел.

Теоретичні відомості

Рекурсивна функція розв'язує прості випадки без подальших викликів і зводить інші випадки до менших підзадач. Кожен виклик займає кадр стека. CPython має обмеження рекурсії, яке можна прочитати через sys.getrecursionlimit(), і не виконує tail-call optimization. Для глибоких структур безпечніше застосувати ітерацію з власним стеком або чергою.

Мемоізація зберігає результат функції для вже обчислених аргументів. functools.lru_cache(maxsize=None) підходить для Fibonaccі, де ті самі значення обчислюються багато разів. cache_info() показує влучання та промахи, а cache_clear() очищає кеш. Для вкладеного аналізу списки спершу перетворюються на кортежі, щоб піддерева стали хешованими ключами кешу.

Постановка завдання

Варіант 6: для вкладених list/tuple чисел рекурсивно обчислити суму, кількість чисел і максимальну глибину вкладення. Реалізувати ітеративну версію з власним стеком і порівняти обидва підходи. За загальними вимогами роботи також реалізувати чистий тришляховий quicksort, рекурсивні обходи дерева, ітеративні BFS/DFS, мемоізований Fibonaccі та його ітеративний аналог.

Хід роботи

1. Створено проєкт lab04 з algorithms.py, main.py, benchmark.py, конфігураціями, каталогом tests, файлами benchmark_results.json і test_results.txt та папкою report.

2. У algorithms.py реалізовано чистий quicksort із випадковим pivot, рекурсивні preorder/inorder/postorder, ітеративні BFS/DFS та memoized Fibonaccі з ітеративним аналогом.

3. Для варіанта 6 analyze_nested повертає (сума, кількість чисел, максимальна глибина). analyze_nested_iterative використовує явний стек, а analyze_nested_memoized перетворює дані на кортежі та кешує результати однакових піддерев.

Структура проєкту

```
lab04/
├── report/ (lab4.docx, lab4.pdf)
├── tests/ (test_quicksort.py, test_tree.py, test_fib.py, test_nested.py)
├── algorithms.py
├── benchmark.py
├── benchmark_results.json
├── main.py
├── mypy.ini
├── pyproject.toml
├── README.md
├── requirements.txt
└── test_results.txt
```

Результати виконання

Для прикладу [1, (2, [3, 4]), ((2, (3, 4)),), []] рекурсивний, кешований та ітеративний алгоритми дали однаковий результат: сума 19, кількість чисел 7, глибина 4.

```
=== Рекурсія та мемоізація: варіант 6 ===
Вкладена структура: (19.0, 7, 4)
Memoized результат: (19.0, 7, 4)
Iterative результат: (19.0, 7, 4)
Fibonacci(30): 832040; hits=28, misses=31
Quicksort: [-1, 0, 3, 3, 5, 8]
BFS: [4, 2, 6, 1, 3, 7]
```

Команду pytest -q --durations=5 виконано: 16 passed in 0.08s. Перевірки mypy та ruff завершилися без зауважень; Python-файли відповідають black.

У benchmark_results.json зафіксовано локальні результати: 100 рекурсивних проходів вкладеної структури — 0.006118 s; ітеративних — 0.004292 s. Для Fibonacci(28) наївна рекурсія — 0.114416 s; memoized — 0.000034 s; 10 000 ітеративних обчислень — 0.012094 s. Це мікровимірювання, результати залежать від середовища.

Висновок

Реалізовано рекурсивні та ітеративні алгоритми для сортування, обходу дерева й обробки вкладених даних, а також мемоізований Fibonacci. Усі 16 тестів пройшли. Кешування значно зменшує повторні обчислення Fibonacci, а власний стек дає змогу аналізувати глибокі структури без перевищення ліміту рекурсії CPython.